ASE 456 • PROJECT PROPOSAL PRESENTATION

DJ King

A live, head-to-head music genre battle game

Team: Red
Daniel Than
Abdullahi Abdirahaman

Presenting: Thursday, September 17

The Problem & The Idea

The Problem

Music apps make listening passive — a playlist plays, nobody competes over it.

Party games (Kahoot, trivia apps) engage groups live, but none are built around music taste itself.

There's no fun, structured way for a group to settle “who has better music taste” in the moment.

What We're Building

A mobile party game where two randomly chosen players battle to pick the best song in a random genre — and the group votes on a winner, live.

Intended Users

College students, roommates, and party hosts — any group of 4–6 friends who want a fast, social icebreaker game for gatherings.

How It Works: The Game Loop

1 Lobby

4–6 players join a session on their phones with a room code.

2 Spin

A genre wheel randomly lands on a music category.

3 Match-Up

Two players are randomly chosen as contestants.

4 Submit

Contestants search & pick their best song before the timer ends.

5 Play It Out

Songs play in full; the picker can stop playback once they've made their case.

6 Vote & Score

The rest of the group votes for the better pick; winner scores a point.

Implementation Approach

FRONTEND

Flutter / Dart

One codebase targeting Android & iOS — course-required cross-platform framework.

REAL-TIME SYNC

Firebase Realtime / Firestore

Syncs lobby state, spinner, timers, and votes live across every phone in the session.

MUSIC DATA

Spotify + YouTube APIs

Spotify for fast search & metadata; YouTube for full-length, licensed playback.

BACKEND LOGIC

Firebase Cloud Functions

Handles room codes, round logic, scoring, and match history.

Architecture: Player App → Real-Time Database → Spotify / YouTube APIs → Playback broadcast to all devices

Software Engineering Planning

Requirements

Functional

- Lobby creation & join codes
- Genre spinner & random matchmaking
- Timed song search & submission
- Full-song playback with manual stop
- Group voting & live scoring

Non-Functional

- Sync latency under ~1–2 seconds
- Stable with 4–6 concurrent devices per session

User Stories

As a host, I want to start a lobby so my friends can join with a code.

As a contestant, I want to search and submit a song before time runs out.

As a song's picker, I want to stop playback once I've made my point.

As a voter, I want to hear both songs and pick the better one.

As a player, I want to see live scores so I know who's winning.

Process, Testing & Risks

Process

Agile — weekly sprints with two-person standups

Version Control

GitHub with feature branches + pull request review

Testing

Unit tests for scoring/round logic + live multiplayer testing between us

Key Risks & Technical Challenges

! Playback licensing

Spotify previews are only 30 sec and not guaranteed; YouTube fills the gap for full-length playback.

! Real-time sync bugs

Vote/timer state must stay consistent across devices under network latency.

! Small test pool

Only 2 team members to test 4–6-player flows — mitigated with mocked/simulated players.

Project Milestones

Week-by-week plan

W4 **Idea finalized & Project Proposal Presentation (20%) — today**

W5 Set up Flutter project structure & Firebase backend

W6 Build lobby creation & join-code flow (UI)

W7 Integrate Firebase real-time sync for lobby state

W8 Build genre spinner UI & random matchmaking logic

W9 Integrate Spotify search API for song submission

W10 **Mid-Project Presentation (40%) — full game loop functional end-to-end**

W11 Integrate YouTube full-song playback + manual stop control

W12 Build voting flow & live scoreboard

W13 Cross-platform testing on Android & iOS; bug fixes

W14 UI polish, animations, edge-case handling

W15 Final testing; prepare live demo + recorded backup

W16 **Final Project Presentation (40%) — completed application**

Team Responsibilities

Initial division of work for the semester — subject to change as development progresses.

Daniel Than

- Flutter/Dart architecture & core app structure
- Firebase real-time sync (lobby, spinner, timers, votes)
- Spotify & YouTube API integration
- Documentation & presentation materials

Abdullahi Abdirahaman

- Game state logic: matchmaking, scoring, playback control
- UI screens & navigation (Flutter widgets)
- Manual testing on Android & iOS
- Documentation & presentation materials

Thank You

Questions?